Problem

We solve the PDE for $x \in [-\pi, \pi)$ (mod 2π), $t \in [0, 1]$, $A = 0.5$:

$$\partial_t u + \partial_x u = -A \sin(x + t), \qquad u(x, 0) = A \cos x$$

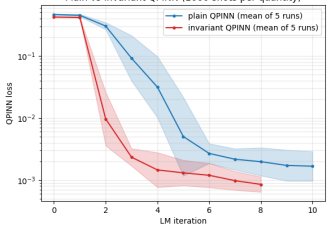
with a **quantum physics-informed neural network (QPINN)** as in **differentiable quantum circuits** (Kyriienko *et al.*, 2021). Many problems have symmetries worth exploiting. Here the PDE and initial condition are unchanged under $(x, t) \to (-x, -t)$, and so is the exact solution $u^* = A \cos x \cos t$: it is **invariant**, so we design an invariant model output.

Approach

- **Analog quantum model.** We consider Hamiltonians on 2 Rydberg atoms as a single layer data re-uploading model: the inputs (x, t) and 4 trainable parameters θ enter as Pauli-Z rotations (exact derivatives by the parameter-shift rule), separated by mixing Hamiltonians with Pauli X (2.6 μs in total). $f_\theta(x, t)$: shot-averaged Pauli Z of atom 0.
- **Hard constraint.** $u_\theta = A \cos x + (1 - \cos t)\, f$ satisfies the boundary conditions automatically, with $f = f_\theta$ (**plain QPINN**) or $f = \frac{1}{2}\left[f_\theta(x, t) + f_\theta(-x, -t)\right]$ (**invariant QPINN**).
- **Training.** Levenberg–Marquardt minimizes the squared PDE residuals R_j at 16 points. Each needed quantity Q (f, R_j, $\partial R_j / \partial \theta_k$) is a weighted sum of model outputs.
- **Randomized estimation.** The shots for Q are split over its terms, sampled separately and summed with weights. This equals drawing a random circuit before every shot, as in randomized encoding[1], and may be easier to run on quantum hardware.

Results

QoolQit emulator, 5 runs/model, 1000 shots/estimate.



Loss per LM step, mean ± std over 5 runs.

	plain	**invariant**
final loss ($\times 10^{-3}$)	1.9 ± 0.9	$\mathbf{0.9 \pm 0.3}$
max error $\lvert u_\theta - u^* \rvert$ ($\times 10^{-2}$)	4.0 ± 1.4	$\mathbf{3.4 \pm 0.3}$
total shots	896k	**704k**

LM stops automatically with the criteria set by us, so the two models run different numbers of iterations.

Why it matters

The invariant model performs clearly better (**−53 % loss, −21 % shots**), and this advantage can carry over to many other problems with symmetries[1].

[1] L. Wang *et al.*, *Fourier symmetrization for geometric quantum machine learning*, arXiv:2610.01874 (2026).

QoolQit version used: v1.4.0

+ Positive aspects

I usually work with qubit models and knew little about analog quantum computing. The Hamiltonian in QoolQit can easily be written in terms of Pauli matrices, and QoolQit gives clear hardware control parameters, which made it easy for me to understand.

– Challenges

Some algorithms may need classical sampling before each shot to decide the parameters of the Hamiltonian, e.g. Canonical quantization of neurons[2], doubly stochastic gradient descent optimizers[3] and Fourier symmetrization for geometric quantum machine learning[1]. QoolQit only runs one fixed Hamiltonian for a number of shots and does not cover this case. I am also not sure how far the hardware supports such operations, so it is unclear how to design the quantum algorithm best suited to the hardware. A method in QoolQit to roughly judge its feasibility (e.g., runtime) on current hardware would be great for users.

[2] A. He, N. Liu and M. M. Wilde, *Canonical quantization of neurons*, arXiv:2607.05000 (2026).

[3] R. Sweke *et al.*, *Stochastic gradient descent for hybrid quantum-classical optimization*, Quantum **4**, 314 (2020).